Machine Learning (2026) 115:16
https://doi.org/10.1007/s10994-025-06914-x

Science-Gym: a simple testbed for AI-driven scientific discovery

Mattia Cerrato[1] · Lennart Baur[1] · Jannis Brugger[2] · Sajjad Shumaly[3] · Nicholas Schmitt[4] · Edward Finkelstein[5] · Selina Jukic[1] · Lars Münzel[1] · Felix Peter Paul[1] · Pascal Pfannes[1] · Benedikt Rohr[1] · Julius Schellenberg[1] · Philipp Wolf[1] · Stefan Kramer[1]

Received: 24 April 2025 / Revised: 20 August 2025 / Accepted: 4 October 2025 /
Published online: 7 January 2026
© The Author(s) 2025

Abstract
Automating scientific discovery has been one of the motivating tasks in the development of AI methods. The task of Equation Discovery (also called Symbolic Regression) is to learn a free-form symbolic equation from experimental data. Equation Discovery benchmarks, however, assume the experimental data as given. Recent successes in protein folding and material optimization, powered by advancements, amongst others, in reinforcement learning and deep learning, have renewed the broader community's interest in applications of AI in science. Nonetheless, these successful applications do not necessarily lead to an improved understanding of the underlying phenomena, just as super-human chess engines do not necessarily lead to improved understanding of chess theory and practice. In this paper, we propose Science-Gym: a new testbed for basic physics understanding. To the best of our knowledge, Science-Gym is the first scientific discovery benchmark that requires agents to autonomously perform data collection, experimental design, and discover the underlying equations of phenomena. Science-Gym is a Python software library with Gym-compatible bindings. It offers seven scientific simulations, which reproduce basic physics and epidemiology principles: the law of the lever, projectile motion, the inclined plane, Lagrangian points in space, brachistochrones, the SIRV model, and the friction force of a droplet. In these environments, agents may be evaluated not only on their ability in e.g. balancing objects on the two beams of a lever, but more importantly on finding equations that describe the overall behavior of the dynamical system at hand.

Keywords Automated discovery · Benchmarking · Symbolic regression · AI for science

Editors: Riccardo Guidotti, Anna Monreale, Dino Pedreschi.

Extended author information available on the last page of the article

1 Introduction

The automated discovery of scientific knowledge has long been a prominent goal in artificial intelligence (AI) research, dating back to the 1970 s (Langley, 1977; Langley et al., 1987). Scientific knowledge manifests itself in various forms, often beginning with a classification and taxonomy of objects, then progressing towards quantitative descriptions that explain and describe a certain system or phenomenon of interest. Among the most effective representations for describing systems are mathematical equations, particularly differential equations. This pursuit has given rise to the field of equation discovery, also known as symbolic regression (Koza, 1994), where the aim is to derive equations from some given experimental data.

Recent years have seen significant advancements in the application of artificial intelligence (AI) to scientific discovery, powered, amongst others, by advancements in natural language processing (NLP) and reinforcement learning (RL). Perhaps the best-known breakthrough in this space is in protein folding (AlphaFold (Jumper et al., 2021)). Large Language Models (LLMs) have otherwise been proven quite useful across a wide array of scientific domains, including material design, drug discovery, and computational chemistry (Microsoft Research AI4Science, 2023). These impacts have encouraged the development of initiatives such as DeepSpeed4Science, which aim to create unified large-scale foundation models to support diverse scientific tasks, including weather modeling and molecular dynamics simulations (Shuaiwen et al., 2023). Equation discovery and symbolic regression, on the other hand, seek to construct human-understandable models that describe system dynamics, typically using ordinary differential equations derived from temporal data. Interpretability is a crucial aspect of these models. If a model cannot be communicated to and understood by the scientific community, it fails to contribute to scientific understanding, even if the technological impact may be significant and a net positive, e.g. in the life sciences. Here, we do not necessarily direct criticism at technological exploitation of the recent breakthroughs in AI, NLP and RL. Rather, we establish as our primary goal to develop methods that may help humans in *understanding* unknown phenomena rather than *optimizing* specific properties of, for example, drugs and materials. We posit that one of the fundamental motivations behind automating scientific discovery is the pursuit of increased interpretability in models that describe natural phenomena. Unlike black-box models, such as deep neural networks, symbolic regression explicitly seeks transparent mathematical representations that can be communicated and understood by human researchers. This clarity allows symbolic regression not only to predict system behaviors accurately, but also to reveal the underlying governing equations that describe those behaviors, enhancing scientific insight and communication. Recent successes in AI-driven scientific discovery–such as protein fold predictions by AlphaFold (Jumper et al., 2021) or material property predictions via deep neural networks–have demonstrated extraordinary predictive performance, but have often fallen short in providing clear interpretative insights into why and how these predictions are made.

The scientific process is often conceptualized as a cyclical and iterative workflow, sometimes referred to as the "science loop" (Fig. 1). This loop represents the continuous and dynamic nature of scientific inquiry, where each phase of the process feeds into the next, fostering ongoing discovery and understanding. The cycle typically begins with the formulation of a *scientific question*, which addresses a specific phenomenon or problem. From this

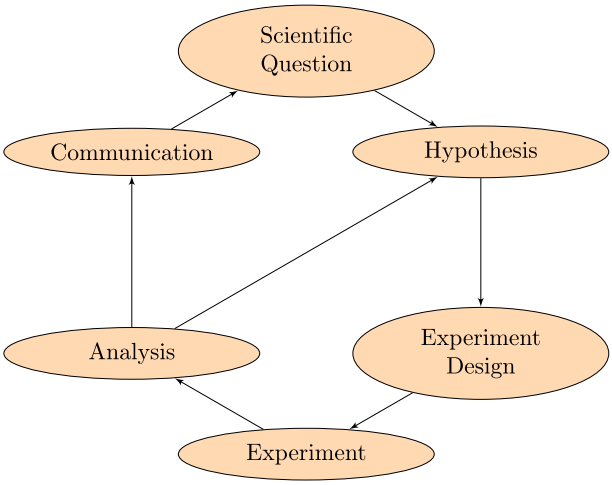

Fig. 1 The "Science Loop": An iterative process of scientific inquiry, encompassing the stages of forming a scientific question, hypothesizing, designing experiments, conducting experiments, analyzing data, and communicating results

question, researchers develop a *hypothesis*, a testable prediction that provides a potential answer or explanation. The next phase involves *experiment design*, where researchers plan and outline the methods and procedures to test the hypothesis. Following this, they conduct the *experiment*, collecting data through systematic observation and controlled investigation. The gathered data are then subjected to analysis, where statistical and computational tools are used to interpret the results and determine whether they support or refute the hypothesis. The findings are subsequently *communicated* to the scientific community through publications, presentations, and discussions, which often lead to new questions and hypotheses, thus restarting the cycle. Thus, establishing clear *levels of autonomy* in discovery agents is another critical consideration (Kitano, 2021). Early systems relied on human-provided data tables, but there has been a shift towards more autonomous approaches. The development of the first robot scientist, Adam, marked a significant milestone by automating cycles of hypothesis generation and testing in functional genomics (Ross et al., 2009). This evolution continues with subsequent generations of robot scientists. Discovery agents now range from passive, supervised and self-supervised learning systems (AlphaFold) to more autonomous models employing active learning (Cohn et al., 1996) and reinforcement learning (Sutton and Barto, 2018).

In this paper, we introduce `Science-Gym`, a novel testbed designed to advance AI-driven scientific discovery and evaluate autonomous discovery agents at every step of the scientific process. The code and library is available at https://github.com/pibborn/science-gym. Science-Gym represents the first benchmark that requires agents to autonomously perform data collection, experimental design, and equation discovery. It is implemented as a Python software library with Gym-compatible bindings, featuring seven scientific simulations that encapsulate fundamental principles of physics and epidemiology: the law of the lever, projectile motion, the inclined plane, Lagrangian points in space, brachistochrones, the SIRV model, and the friction force of a droplet. These environments allow for the evaluation of agents not only on task performance but also on their ability to uncover underlying equations that describe system behaviors. Furthermore, successful discovery agents will have to explore the experimental environment on their own. `Science-Gym` seeks to facilitate the scientific practice of developing and benchmarking automated discovery models: it

seeks to bridge the interpretability gap left by opaque predictive models, enabling scientists to construct mental models and physical intuitions grounded in transparent mathematical relationships. In this context, Science-Gym represents a potential step forward by providing an accessible benchmark that explicitly rewards agents not just for predictive performance, but also for discovering comprehensible equations and gathering the relevant experimental data to do so. This dual emphasis makes it uniquely suited as a testbed for future AI methods, aiming not just at automation, but at enhancing scientific understanding and education. One important caveat is that not all systems may be neatly summarized by one "master equation" or system of equations: This is particularly the case for highly complex, poorly understand systems, systems where the number of variables vastly outnumbers the number of instances, and systems with noisy and potentially unobservable variables, as frequently encountered in biology (Krishnan et al., 2007). We leave addressing such epistemiological and, on the technical side, knowledge representation topics to future work.

The remainder of this paper is structured as follows: Following an account of related work and background information (Sect. 2 and 3), we give a detailed overview of `Science-Gym` (Sect. 4), its building blocks and physics and epidemiology benchmarks (Sect. 5), including the currently defined state/observation and action spaces and reward functions. Section 6 sketches research directions with `Science-Gym` and presents some baseline results with the framework, before we conclude and give an outlook on future improvements in Section 7.

2 Related work

The field of symbolic regression and equation discovery has seen significant advancements, particularly in the development of benchmark datasets and evaluation frameworks. These resources are critical for assessing the performance and robustness of new methods, ensuring that they can accurately and efficiently discover underlying mathematical relationships in various types of data. Below, we describe some of the most notable benchmarks and datasets, highlighting their unique features and emphasizing distinctions from the Science-Gym framework. Our aim here is not to summarize the long history of equation discovery and symbolic regression, especially as it regards methodologies. We refer the interested reader to recent survey papers (Kramer et al., 2023; Makke and S., 2024), whereas here we will focus on existing benchmarks in the space of symbolic regression and scientific discovery more broadly.

2.1 Symbolic regression benchmarks

The Feynman Symbolic Regression Database (FSRD) is a comprehensive collection of physics-inspired equations derived from the Feynman Lectures on Physics. This dataset is designed to provide realistic and challenging problems for testing symbolic regression algorithms. Each equation in the FSRD represents a fundamental physical law, making it an ideal benchmark for evaluating the ability of symbolic regression methods to rediscover known scientific principles from data (Udrescu and Tegmark, 2020). While FSRD focuses on rediscovering physical laws from predefined equations, Science-Gym extends this by requiring agents to autonomously perform data collection and experimental design. Sci-

ence-Gym includes simulations such as projectile motion and the SIRV model, providing a more interactive and dynamic environment for testing AI-driven scientific discovery.

The Penn Machine Learning Benchmarks (PMLB) is a comprehensive collection of datasets designed for evaluating machine learning algorithms, including symbolic regression methods. PMLB includes datasets from various domains, such as biology and physics, categorized into easy, medium, and hard difficulty levels. This diverse collection is used to assess the performance of symbolic regression methods across different types of data and problem complexities (Olson et al., 2017). PMLB provides a wide range of datasets for traditional machine learning tasks, but it does not focus specifically on the autonomous experimental design and data collection aspects. Science-Gym, on the other hand, challenges AI agents to not only solve equations but also to engage in the full scientific process, including experimentation and hypothesis testing.

SRBench is an ongoing project aimed at maintaining a comprehensive and up-to-date benchmark for symbolic regression methods. It includes a wide array of symbolic regression techniques, from classic genetic programming approaches to modern machine learning-based methods. SRBench incorporates datasets from the Feynman equations, the ODE-Strogatz repository, and other sources, providing a robust framework for evaluating the performance and generalizability of symbolic regression algorithms (La Cava et al., 2021). While SRBench provides a valuable resource for benchmarking symbolic regression methods, it primarily focuses on the computational aspects of equation discovery. In contrast, Science-Gym emphasizes the broader scientific discovery process, requiring agents to autonomously design experiments and gather data in dynamic simulation environments.

The ODE-Strogatz Repository includes datasets based on ordinary differential equations (ODEs) inspired by the work of Steven Strogatz. These datasets are particularly useful for testing symbolic regression methods on dynamic systems, where the goal is to discover underlying differential equations that govern the system's behavior. The ODE-Strogatz datasets help evaluate the ability of symbolic regression techniques to handle complex, time-dependent data (Strogatz, 1994). Similarly to SRBench, this data collection does not especially support agents in learning their own data collection strategies.

2.2 Other discovery benchmarks

Wang et al. (2022) is a publicly accessible reinforcement learning platform aimed at evaluating AI agents' skills in performing scientifically grounded reasoning tasks within virtual lab scenarios. It comprises 30 distinct text-based tasks. Agents are assessed based on their ability to successfully complete these tasks within a predefined number of simulation steps.

DiscoveryWorld (Jansen, 2024) is an interactive, open-source simulation environment designed specifically for benchmarking AI agents across the entire scientific discovery pipeline, covering stages from generating hypotheses and planning experiments to executing them and analyzing the results. The environment resembles a two-dimensional, grid-based role-playing game, offering a series of missions and subtasks that agents must accomplish sequentially to progress through the discovery process.

BoxingGym (Gandhi et al., 2025) introduces a benchmark aimed at evaluating AI agents' capabilities in automated scientific discovery, encompassing both experimental design and model formulation. The benchmark comprises ten environments, each modeled as a generative probabilistic system derived from diverse scientific domains such as psychology

and ecology. Agents interact with these environments to perform experiments and propose explanatory models. Evaluation metrics include expected information gain (EIG) to assess the informativeness of experiments and the predictive accuracy of the proposed models. The authors find that current large language models (LLMs), including GPT-4o, face challenges in both designing informative experiments and formulating accurate models, even when supplemented with explicit statistical tools.

3 Background and preliminaries

In this section, we briefly review the two foundational components underlying Science-Gym: reinforcement learning (RL) and symbolic regression (SR). Together, these fields provide a powerful foundation for autonomous scientific discovery, where agents learn to design experiments, collect data, and uncover interpretable models that describe observed phenomena.

3.1 Reinforcement learning

Reinforcement learning (RL) is a framework for sequential decision-making where an agent interacts with an environment to learn behaviors that maximize cumulative reward. Formally, an RL environment is modeled as a Markov Decision Process (MDP), defined by a tuple $(\mathcal{S}, \mathcal{A}, P, R, \gamma)$, where:

- $\mathcal{S}$ is the set of environment states,
- $\mathcal{A}$ is the set of available actions,
- $P(s' \mid s, a)$ is the transition function, giving the probability of transitioning to state s' from state s after taking action a,
- $R(s, a)$ is the reward function, specifying the scalar feedback for taking action a in state s,
- $\gamma \in [0, 1)$ is a discount factor for future rewards.

The agent seeks to learn a policy $\pi(a \mid s)$ that maximizes the expected return:

$$\mathbb{E}_\pi \left[\sum_{t=0}^{T} \gamma^t R(s_t, a_t) \right],$$

where T is the length of an episode, which may be fixed or determined by a terminal state condition.

In RL, an *episode* refers to a finite sequence of interactions beginning from an initial state and terminating either after a fixed time horizon or upon reaching a terminal condition. In the context of Science-Gym, an episode is naturally mapped to a single scientific experiment.

As we will see in Sect. 4, each environment in Science-Gym defines:

- An initial experimental configuration, corresponding to the initial state s_0.

- A space of experimental interventions or manipulations, corresponding to the action space $\mathcal{A}$.
- A dynamic simulation governed by physical or epidemiological laws, which determines the transition dynamics of the simulation and environment.
- An observation or measurement outcome, forming the new state s_{t+1}.
- A task-specific reward function $R(s_t, a_t)$, e.g., whether the experiment leads to equilibrium or minimizes travel time.

Thus, conducting an experiment in Science-Gym is equivalent to running an RL episode. The agent interacts with the simulation through one or more actions, observes the resulting system behavior, and receives feedback in the form of a reward. In most environments, the episode ends after a single or small number of steps, reflecting the completion of a scientific trial. The episodic formulation supports the notion of *trial-and-error experimentation*, making Science-Gym well-aligned with the iterative nature of scientific inquiry. Moreover, this episodic structure enables agents to autonomously explore different configurations of a system, gather data, and subsequently apply symbolic regression to infer governing equations. This use of RL is distinct from prior work, which primarily applies RL for equation generation rather than for the design and execution of scientific experiments themselves.

3.2 Symbolic regression

Symbolic regression is a form of supervised learning that searches for mathematical expressions best fitting a given dataset. Unlike standard regression, where the model structure is fixed (e.g., linear or polynomial), symbolic regression aims to simultaneously discover both the structure and parameters of the equation. This is typically done using a predefined set of operations $\mathcal{F}$ (e.g., `+`,`-`, `*`, `/`, `sin`, `exp`) and terminals $\mathcal{T}$ (e.g., input variables and constants).

Given a dataset $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^n$, the goal is to find an expression f such that the error

$$\mathcal{L}(f) = \frac{1}{n} \sum_{i=1}^{n} (y_i - f(x_i))^2$$

is minimized, subject to constraints on complexity or interpretability. The resulting models are typically represented as a expression trees or other formalisms for expressions that can be directly interpreted and analyzed.

Symbolic regression is especially attractive for scientific applications, because it outputs human-readable equations, which can be related to known laws of physics, chemistry, or biology.

3.3 RL for symbolic regression

Recent works have explored combining RL with SR to guide the symbolic model construction process (Kramer et al., 2023). In such cases, the RL agent learns to sequentially construct expressions (e.g., building trees token-by-token), where rewards reflect improvements in predictive accuracy or simplicity.

While these methods treat symbolic regression as the ultimate goal of the RL process, they assume access to a fixed dataset. In contrast, Science-Gym extends this paradigm by embedding SR within a full scientific process: The agent must first design and execute experiments to generate informative data, and only then applies symbolic regression to extract governing equations. This introduces a new research challenge where RL is not only used to build models, but also to explore and shape the data collection process itself–a step toward more autonomous and interpretable AI systems in science.

4 Overview of `Science-Gym`

The Science-Gym library includes a variety of classical physics problems (plus one from epidemiology) that serve as environments for testing AI-driven scientific discovery (see Fig. 2). Its development was driven by a fundamental question: What is the minimal information that could be given to an agent (or algorithm) to independently discover scientific knowledge? We determined that the concept of an experiment was essential. An experiment provides a structured starting situation and a clear completion condition, aligning well with the definition of an episodic Markov Decision Process (MDP). In an MDP, the state space represents the evolving state of an experiment over time. This state can be expressed in various forms, such as a table of observational data or raw pixel data simulating visual perception. The action space, which encompasses all possible experimental actions the agent can take, will differ based on the phenomenon at hand. For instance, in `Lagrange-v1`, the agent can place a satellite into space. In `Lever-v1`, the agent can place items on a two-beam scale. This flexibility allows the agent to interact dynamically with the environment, exploring different hypotheses and learning from the outcomes. Furthermore, while rewards are not inherently provided by nature, incorporating them in our simulations enables the development of baseline performance metrics and facilitates the training of reinforcement learning algorithms. We have designed rewards that may provide directions towards discovery, that is, towards exploring interesting states of the environment such as equilibria. Our rationale is that this is guiding it towards successful scientific discovery. Nonetheless, we observe that maximizing the given rewards in these environments is relatively straightfor-

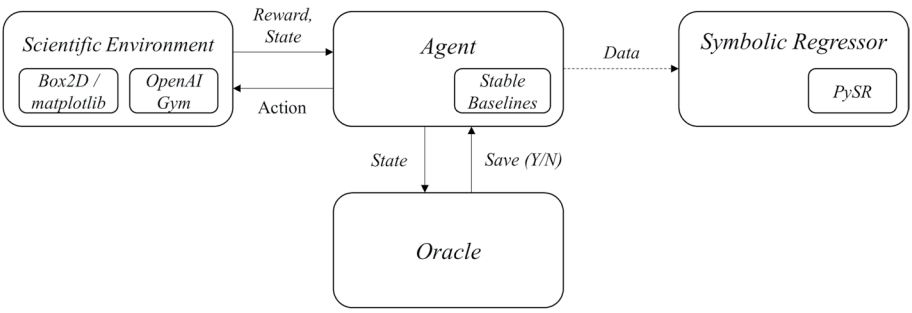

Fig. 2 Overview of framework with integrated software libraries and the "full support" scenario (see Section 4.2 for a detailed explanation). A reinforcement learning agent conducts experiments and gathers data, if they are deemed interesting by an oracle, for characterization by an equation discovery/symbolic regression system

ward, and we encourage researchers interested in scientific discovery to provide evaluation on more challenging contexts, such as with noisy or no rewards (see Section 4).

The Science-Gym library is built to be compatible with the Gym API style, chosen for its simplicity and widespread adoption in the reinforcement learning community. This design choice ensures compatibility with a multitude of existing reinforcement learning frameworks and tools, fostering ease of integration and experimentation. The environments are designed to be modular and flexible, allowing users to easily modify parameters and customize the experiments to suit their specific research needs.

However, the library has its limitations. One significant limitation is the lack of explicit mechanisms for discovering new observational tools (i.e. columns in the experimental data table) or a custom domain-specific language, such as those found in the CLEVR and CLEVRER visual reasoning tasks (Johnson et al., 2017; Yi et al., 2020). These limitations restrict the ability of agents to fully emulate the creative and hypothesis-generating aspects of human scientific discovery. Future versions of `Science-Gym` will focus on incorporating these features to enhance the capability of agents to autonomously generate and test novel hypotheses, further bridging the gap between artificial and human-driven scientific discovery.

4.1 APIs and library structure

`Science-Gym` is structured into two main classes that handle physical simulation and representing the scientific phenomena available. These are the `Simulation` and `Problem` classes, respectively. We also employ the `sympy` library to represent both candidate `equations` and actual solutions to the problems, that is, the set of equations that represents an acceptable description of the simulated phenomena.

4.1.1 Simulation

The physical simulation environments are structured around an abstract interface named `SimulationInterface`, which extends the `gym.Env` class from the OpenAI Gym framework. This interface defines essential methods required to integrate simulation models with reinforcement learning algorithms, such as `step()`, `reset()`, and methods to retrieve the current state and define state and action spaces.

An example implementation of this interface is the `OrbitEnvironment`, which leverages the Box2D physics engine through the Python bindings (`pybox2d`) to accurately simulate gravitational dynamics between celestial bodies. Objects within this environment, specifically instances of the `Planet` and `Satellite` classes, are modeled as dynamic entities whose physical interactions are computed by Box2D. The environment also utilizes additional libraries, including `NumPy` for numerical computations, `pygame` for rendering visualizations, and OpenCV (`cv2`) for auxiliary image processing tasks. Actions taken in this environment correspond to forces applied to the satellite, and observations are structured representations capturing positions, velocities, and physical parameters of the simulated bodies. The modular and dependency-rich architecture of this implementation enables comprehensive experimentation with physics-based reinforcement learning scenarios.

4.1.2 Problem

The class `ProblemInterface` serves as an abstraction that integrates simulation environments into structured problem definitions for symbolic regression tasks. It encapsulates an instance of the previously described `SimulationInterface`, delegating core environment interactions (e.g., `step()`, state retrieval, and action space definitions) directly to the underlying simulation through method delegation. Dependencies include numerical libraries (`NumPy` and `pandas`) and symbolic representation functionalities provided by the `sciencegym` library through the `Equation` class. The `ProblemInterface` explicitly defines methods crucial to symbolic regression, such as `evaluation()`, which computes prediction errors between candidate equations and known solutions, and abstract methods (`solution()`, `validate_context()`) intended for implementation by derived classes.

As a concrete example, the class `Problem_Lagrange` implements this interface to represent a specific symbolic regression problem involving the Lagrange point in a three-body gravitational scenario. It defines relevant variables such as masses and distances between celestial bodies, and explicitly provides the known analytical solution: $y = \frac{\sqrt{3}}{2} distance(b_1, b_2)$, where b_1 and b_2 are the two other bodies in the system. This structured representation simplifies benchmarking symbolic regression algorithms against known physical equations.

4.1.3 Equation

The class `Equation` provides a structured representation of symbolic mathematical expressions, leveraging the `sympy` library for symbolic computation. When initialized from a string, it automatically parses the expression, identifies and manages symbolic variables, and enables efficient numerical evaluation through vectorized operations using `numpy` and `pandas`. The `evaluate()` method allows straightforward evaluation of the symbolic equation on tabular data provided as a `pandas` DataFrame, extracting only the necessary variables for computation. Additionally, the class offers built-in functionality for measuring the complexity of an expression, defined by the number of symbolic operations involved, and supports symbolic equality comparisons, simplifications, and seamless conversion back to `sympy` objects for further manipulation.

4.1.4 Agents and exploration

We offer implementations of RL agents through the `StableBaselinesAgent` class, which is itself an implementation of the `Agent` from the `Stable-Baselines3` library (Raffin et al., 2021). Stable baselines 3 is a reinforcement learning library that prioritizes reproducibility of results and clarity of implementation. We employ it in `Science-Gym` to train and evaluate RL agents on Science-Gym environments using modern, off-the-shelf algorithms such as Soft Actor-Critic (SAC), Proximal Policy Optimization (PPO). The agent supports training via vectorized Gym environments (`DummyVecEnv`) and allows for adjustment of random exploration parameter and reward normalization, if available. The class provides a complete RL example training loop (`train_loop`) that orchestrates environment setup, model creation, training, and periodic evaluation against test environments.

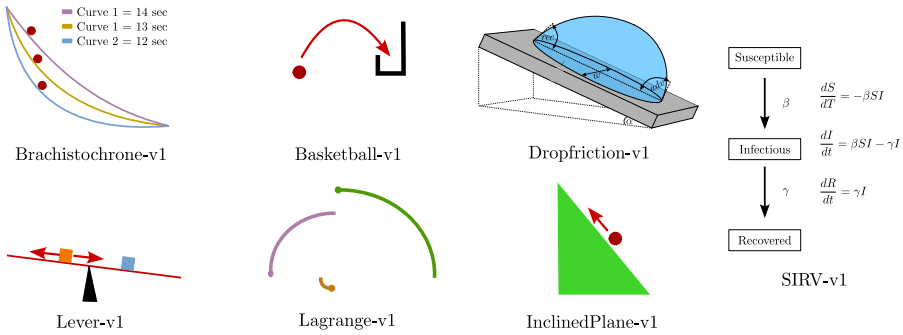

Fig. 3 `Science-Gym` is a collection of 7 physical simulations with `Gym`-compatible APIs. Summary image for the SIRV model by Luz et al. (2010) and Dropfriction is adapted from Li et al. (2023)

Table 1 Result equations for each problem in Science-Gym

Problem	Equation	Alternative	Differential form
Brachistochrone	$x(\theta) = R(\theta - \sin(\theta))$ $y(\theta) = R(1 - \cos(\theta))$	$T = \int_A^B \frac{\sqrt{1+(y')^2}}{\sqrt{2gy}}\, dx$	
Law of the lever	$F_1 d_1 = F_2 d_2$	$\tau_1 = \tau_2$	$\frac{d\tau}{dx} = 0$
Projectile motion	$x(t) = v_0 \cos(\theta) t$ $y(t) = v_0 \sin(\theta) t - \frac{1}{2} g t^2$	$y = x \tan(\theta)$ $- \frac{g x^2}{2 v_0^2 \cos^2(\theta)}$	$\frac{d^2 x(t)}{dt^2} = 0$ $\frac{d^2 y(t)}{dt^2} = -g$
Inclined plane	$F_{\parallel} = mg \sin(\theta)$	$F_{\perp} = mg \cos(\theta)$	$\frac{d^2 x(t)}{dt^2} = g \sin(\theta)$ $\frac{d^2 y(t)}{dt^2} = -g \cos(\theta)$
Lagrangian points	$F_{gravity} = F_{centripetal}$	$L_x^4 = \frac{R}{2} - d$ $L_y^4 = \frac{\sqrt{3}}{2} \cdot R$	
Drop friction	Ground truth unknown		

The `evaluate()` method runs a single episode and computes the cumulative reward, supporting deterministic policy execution. Testing is performed via `test_loop()`, which runs multiple episodes, collects high-performing trials, and updates structured data tables accordingly–enabling downstream equation discovery. To interface with environments using complex observation spaces, the class includes utilities to convert `Dict`-based state representations into flattened vectors suitable for stable-baselines-compatible models. The modular design also supports model persistence via saving and loading, and supports logging metrics such as average test rewards and success rates (Biewald, 2020).

Table 2 Specification of physics and epidemiological problems

Environment	Parameters	State space	Action space	Reward	Rendering
Brachstochrone-v1	Step size $s \in (0,1)$	$y_t = y_{t-1} + \delta y_t \cdot s$	δy_{t+1}	$T_{opt} - T_T$	matplotlib
Basketball-v1	Gravity g Basket positions (b_x, b_y) Ball positions (o_x, o_y) Ball densities d	(b_x, b_y),(o_x, o_y), d	F_b	Score	Box2D
Lever-v1	Gravity g Object densities (d_1, d_2) Object masses (m_1, m_2)	d_1, d_2,m_1, m_2,τ_1	τ_2	$1/\alpha$	Box2D
SIRV-v1	Compartment sizes S, I, R	S_0, I_0, R_0	V_0	$\frac{\lvert I_1 - I_0 \rvert}{I_0}$	None
Lagrange -v1	Masses m_i of bodies	(x_0, y_0) for the two bodies Masses (m_1, m_2)	(x_0, y_0) for the satellite	$\lvert\lvert(x_0, y_0) - (x_T, y_T)\rvert\rvert^2$	matplotlib
InclinedPlane-v1	Gravity g Angles (α_1, α_2) Masses (m_1, m_2)	g, α, m	$\lvert F_\parallel \rvert$	$\lvert\lvert F^\star - F_\parallel \rvert\rvert^2$	Box2D
Dropfriction-v1	Drop length/width d, w Advancing/receding contact angle adv, rec Velocity vel	d, adv, rec, vel, w	plane tilt angle α, time t	See Fig. 4	None

5 Scientific problems in Science-Gym

In the following, we give a brief description of the scientific problem simulated by each environment available in `Science-Gym`. A graphical representation of each environment is available in Fig. 3. A summary of possible mathematical descriptions of the underlying problems may be found at Table 1, whereas the simulation parameters, state space, action space and rewards are summarized in Table 2.

5.1 The brachistochrone problem

`Brachistochrone-v1` **.** The brachistochrone problem, first posed by Johann Bernoulli in 1696, is a classic problem in the calculus of variations. It involves finding the shape of a curve $y(x)$ down which a particle will slide under the influence of gravity from a point A to a lower point B in the shortest time. The solution to this problem is a cycloid, given by the parametric equations:

$$x(\theta) = R(\theta - \sin(\theta))$$

$$y(\theta) = R(1 - \cos(\theta)),$$

where R is the radius of the generating circle, and θ is the parameter. The differential equation representing the brachistochrone problem is derived from minimizing the integral of the time functional:

$$T = \int_A^B \frac{\sqrt{1 + (y')^2}}{\sqrt{2gy}}\, dx$$

`Brachistochrone-v1` is implemented by having an agent output the *y*-axis position of points that are given equispaced on the *x* axis. Naturally, this will only approach a cycloid in the limit that the number of points is infinite. This provides an additional challenge in terms of discovery. The reward function compares the time it takes an object to descend the curve, and compares it with the one it would take for the optimal result.

5.2 The law of the lever

`Lever-v1` **.** The law of the lever is a fundamental principle of mechanics discovered by Archimedes. It states that the torque τ exerted by two weights on opposite sides of a pivot point (fulcrum) is proportional to their distances from the pivot. The law is expressed as:

$$\tau_1 = \tau_2 \implies F_1 \cdot d_1 = F_2 \cdot d_2,$$

where F_1 and F_2 are the forces applied at distances d_1 and d_2 from the pivot, respectively. The equilibrium condition for the lever can be written as a differential equation involving the forces and distances:

$$\frac{d\tau}{dx} = 0$$

`Lever-v1` is implemented as a physical simulation with (Catto, 2021). A box of certain density is placed on the right side at some position, and the agent's task is to place another box of given density on the left side. The implemented reward function is scaled according to the tilt of the lever system, where angles closer to 0 are rewarded more highly.

5.3 Projectile motion

`Basketball-v1` **.** Projectile motion involves analyzing the motion of an object projected into the air, subject to acceleration due to gravity. The problem is modeled by the following set of differential equations:

$$\frac{d^2x(t)}{dt^2} = 0$$

$$\frac{d^2y(t)}{dt^2} = -g,$$

where $x(t)$ and $y(t)$ are the horizontal and vertical positions of the projectile at time t, respectively, and g is the acceleration due to gravity. The initial conditions are given by the initial velocity components $v_0 \cos(\theta)$ and $v_0 \sin(\theta)$. Solving these equations yields:

$$x(t) = v_0 \cos(\theta)t$$

$$y(t) = v_0 \sin(\theta)t - \frac{1}{2}gt^2,$$

where v_0 is the initial velocity and θ is the angle of projection. `Basketball-v1` is implemented as a (Catto, 2021) solo basketball game. A ball of some density is placed at some random position, and the agent's task is to give the force vector that will result in scoring. Successful shots are counted by the underlying reward function.

5.4 The inclined plane

`Plane-v1` **.** The inclined plane is a simple machine that involves analyzing the forces acting on an object sliding down the plane due to gravity. The problem is described by the differential equations for the parallel and perpendicular components of gravitational force:

$$F_{\parallel} = mg \sin(\theta)$$

$$F_{\perp} = mg \cos(\theta),$$

where m is the mass of the object, g is the acceleration due to gravity, and θ is the angle of inclination. The equations of motion for the object are:

$$\frac{d^2x(t)}{dt^2} = g \sin(\theta)$$

$$\frac{d^2y(t)}{dt^2} = -g \cos(\theta)$$

`InclinedPlane-v1` is implemented with (Catto, 2021). At the start of the experiment, a ball of a certain density is placed at some point of the plane. The task of the agent is to give the force vector that keeps the ball in a steady position, as the gravity would have it roll down eventually. The reward is inversely proportional to how much the ball moves, in either direction.

5.4.1 Lagrangian points in space

`Lagrange-v1` **.** Lagrangian points are positions in space where the gravitational forces of two large bodies, such as the Sun and the Earth, balance the centripetal force felt by a much

smaller object of negligible mass, such as a satellite. There are five Lagrangian points, (L^1 through L^5). Given the distance R between the two large bodies, the position of L^4 and L^5 is given by the remaining vertex of an equilateral triangle that passes through the two bodies. If R is the distance between the two bodies, m_1 and m_2 their respective masses so that one may define $d = \frac{m_2}{m_2+m_1} \cdot R$, the coordinates of L^4 and L^5 are given by:

$$L_x^4 = L_x^5 = \frac{R}{2} - d \quad L_y^4 = \frac{\sqrt{3}}{2} \cdot RL_y^5 = -\frac{\sqrt{3}}{2} \cdot R$$

on a 2D plane centered at the center of mass for the overall system.

`Lagrange-v1` is implemented via a `matplotlib` visualization of the path of the satellite after one full cycle of the system. If a satellite has been placed at a Lagrange point, the balance of the gravitational forces will result in the satellite being in the same exact spot. The reward given by the environment is inversely proportional to the distance the satellite will have traveled.

5.4.2 SIRV model

`SIRV-v1` **.** The SIRV model is an extension of the classic SIR (Susceptible-Infectious-Recovered) model used in epidemiology to simulate the spread of infectious diseases. The SIRV model incorporates an additional compartment, V, representing vaccinated individuals. The model divides the population into four compartments: Susceptible (S), Infectious (I), Recovered (R), and Vaccinated (V). The dynamics of the model are governed by a set of differential equations:

$$\frac{dS}{dt} = -\beta SI - \mu S \frac{dI}{dt} = \beta SI - \gamma I$$

$$\frac{dR}{dt} = \gamma I \frac{dV}{dt} = \mu S$$

Here, β represents the transmission rate of the disease, γ is the recovery rate, and μ is the vaccination rate. The susceptible individuals (S) can become infected (I) through contact with infectious individuals at a rate proportional to β. Infected individuals recover at a rate γ and move to the recovered compartment (R). Susceptible individuals can also be vaccinated at a rate μ and move to the vaccinated compartment (V), where they are assumed to be immune to the infection. The SIRV model is particularly useful for studying the impact of vaccination programs on the spread of infectious diseases.

`SIRV-v1` is currently implemented without a rendering function. The agent is given the sizes of the S, I and R compartments at time step 0, and it then decides on the size of the V compartment. The discovery-directed reward is given as $\frac{|I_1 - I_0|}{I_0}$ to limit the simulation time.

5.4.3 A real-world task

`Dropfriction-v1` . The droplet friction problem is an analysis of the movement dynamics of liquid droplets sliding down an inclined plane. While the equation for frictionless motion of a body on the inclined plane is well-known, predicting the movement of liquid drops is an open problem in materials science, and a ground truth equation is currently not available. In particular, generalizing to different materials for both droplets and planes is challenging: we refer the interested reader to a recent paper (Li et al., 2023), which measures the friction force of a droplet sliding down a tilted plane for 23 material-liquid combinations.

In contrast to the other problems in `Science-Gym`, the equations proposed so far cannot completely solve the problem. The Furmidge-Kawasaki equation (Kawasaki, 1960; Furmidge, 1962) suggests that the friction force F_f may be approximated by the equation $F_f = w\gamma k(\cos(Rec) - \cos(Adv))$, where γ is the surface tension and k is a shape factor – for the other variables, we refer to Table 2. The "state of the art" in predicting the friction force of a droplet on a plane is $F_f = F_0 + \beta w \eta \cdot vel$ (Li et al., 2023) with β as friction coefficient and η viscosity as material-specific variables.

For `Science-Gym`, we use the experiments for a glycol-ethylene droplet on a teflon-gold surface. While we plan to add all liquid-surface combinations in the future, due to data confidentiality we train a multi-layer perceptron (MLP) to approximate the data coming from 77 experiments with 9744 records (rows) in total. The task in this problem is to select the tilt angle and measuring time for an experiment. The environment, simulated by the MLP, returns the dynamics of the droplet for the experimental conditions chosen by the discovery agent. The agent gets a reward close to 1 if it selects a measuring time inside the data measured in Li et al. (2023) and is not close to a previously measured point. For clarity, we give in Fig. 4 the definition of reward function and the contours of the sampling area that returns a non-negative reward.

6 Research with `Science-Gym`

In this section, we give a few suggestions and pointers about how `Science-Gym` may be employed in research on autonomous discovery systems.

$$\text{reward}(a) = \begin{cases} -1, & \text{if } \alpha \notin [-1.7, 1.7] \\ -1, & \text{if } t \notin [-1.3, e^{-\alpha}] \\ 1, & \text{if } \mathcal{A}_{\text{past}} = \varnothing \\ \min(|a - a_i|_{l1}, 1)\ \forall a_i \in \mathcal{A}_{\text{past}}, & \text{otherwise} \end{cases}$$

$$\text{if reward}(\alpha, t) > -1 : \ \mathcal{A}_{\text{past}} \leftarrow (\alpha, t) \cup \mathcal{A}_{\text{past}}$$

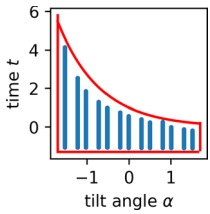

Fig. 4 (left) Reward function for Dropfriction-v1. (right) A graphical representation of the contours of the sampling area that returns a non-negative reward. The blue bars represent measurements from Li et al. (2023)

6.1 States and rewards

In Science-Gym, the complexity of tasks given to an agent can be modulated along two primary dimensions: the type of observations provided – that is, the complexity of the state space – and the nature of the rewards. By varying these dimensions, researchers can create environments of varying difficulty levels, which will require different levels of autonomy.

6.1.1 Observations

The type of observations provided to the agent significantly influences the complexity of the task. Observations can be provided in two main forms: either as tables or raw pixels. When data is given as tables, the agent receives structured observational data. These tables describe the initial and final experimental results, alongside every intermediate step. This representation makes it significantly easier for an agent to identify patterns, draw conclusions, and formulate hypotheses. On the other hand, the tabular format reduces the complexity of data interpretation and visual parsing, allowing researchers to focus more on the efficiency of the discovery process. In contrast, when observations are provided as raw pixel data, the agent must process and interpret visual information to understand the experimental setup and outcomes. This form of observation mimics human visual perception and adds a layer of complexity, as the agent must employ image processing techniques to extract meaningful data.

6.1.2 Rewards

The nature of rewards also plays a crucial role in shaping the agent's behavior and learning process. Rewards in `Science-Gym` may be either i) directly given as a form of directing the agent towards interesting states; ii) given, but noisily; iii) or not given at all. The rewards implemented in `Science-Gym` explicitly direct the agent towards regions of the state space that are relevant for the discovery problem at hand. As an example, rewards are higher when the lever in `Lever-v1` is balanced or the projectile in `Basketball-v1` hits the target. The main motivation for the inclusion of these rewards is to provide strong baselines to interested researchers: Arguably, there are no short-term rewards provided by nature for the understanding of certain phenomena. As an alternative, we provide simple ways to

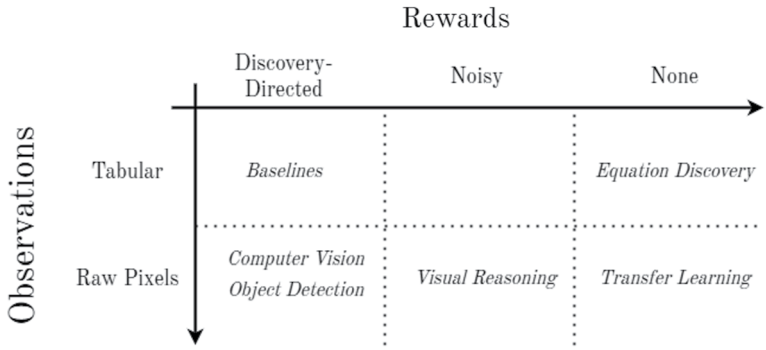

Fig. 5 Contexts of increasing difficulty in Science-Gym, with recommendations about which techniques and fields might benefit from investigating them

provide noisy rewards to agents. Nonetheless, the most challenging (and realistic) scenario that should be tackled with `Science-Gym` is without any rewards. In such a scenario, an agent would have to discover interesting states, like equilibria, by curiosity autonomously, and then aim to reproduce them and characterize them by equations.

6.2 Context and autonomy

The concept of contexts in Science-Gym is crucial for understanding the level of information provided to an agent performing scientific discovery. Contexts define how much support the agent receives in forming hypotheses and designing experiments. By varying the contexts, researchers can evaluate the capabilities of their AI agents under different levels of difficulty (see Fig. 5). Below, we describe several contexts of increasing difficulty:

6.2.1 Context 1: full support

In this context, the agent is given maximal support to facilitate scientific discovery. This includes:

6.2.1.1 Rewards Providing explicit rewards for correct hypotheses and successful experiments. This significantly simplifies the agent's task by giving clear feedback on its actions.

6.2.1.2 Observational data tables Supplying detailed tables that describe initial and final experimental results. This structured data allows the agent to easily identify patterns and draw conclusions.

6.2.2 Context 2: partial support

Here, the agent receives moderate support, making the task more challenging. This context includes:

6.2.2.1 Noisy rewards Providing rewards, but with less frequency or only for major milestones. This encourages the agent to explore more and develop a deeper understanding of the experimental setup.

6.2.2.2 Observational data Offering raw observational data without pre-processed tables. The agent must process and interpret this data to form hypotheses and design experiments.

6.2.3 Context 3: minimal support

In this context, the agent is given minimal support, requiring it to perform most of the scientific discovery independently:

6.2.3.1 No rewards Removing explicit rewards. The agent must rely solely on its ability to interpret the results of its actions to determine success.

6.2.3.2 Raw data Only raw data is provided in the form of pixels. The agent must extract relevant information and derive conclusions without guidance.

6.2.4 Context 4: full autonomy

The most challenging context requires the agent to operate with complete autonomy:

6.2.5 Self-generated rewards

The agent must develop its own criteria for success and self-generate rewards based on its progress.

6.2.6 No direct observations

The agent must infer experimental results from indirect measurements or incomplete data.

These varying contexts allow researchers to systematically test and improve the robustness and adaptability of automated discovery techniques. We therefore recommend researchers interested in `Science-Gym` to clarify which context has been employed in their reports and publications. We report experiments with noisy and sparse rewards (Context 2) in Sect. 6.4, and provide a baseline for Context 1 in the next subsection.

6.3 A baseline for the full support scenario

In this section, we report a simple strategy that may be employed to straightforwardly obtain equations from the presented environments. Our point here is not to present `Science-Gym` as a solved task; rather, we wish to stress the centrality of contexts in understanding which level of autonomy has been achieved by some discovery system. For this purpose, we present here in the following a methodology that relies on readily available, off-the-shelf components and manages to find descriptive equations for Context 1 (Full Support) in some environments (see also Fig. 2). This technique relies on the discovery-guided rewards available in each environment, and we dub it Threshold-and-Save. It may be summarized as follows:

1. Use an off-the-shelf RL algorithm to explore and collect rewards from a given environment;
2. Save all episodes, including initial and final values for the state space, in which the cumulative reward $R_t \geq R^*$, where R^* is some threshold;
3. Employ an off-the-shelf equation discovery system to obtain the underlying equations representing the phenomenon.

Table 3 Results for Threshold-and-Save on different environments, employing `PySR` for the extraction and `SAC` for the exploration of the environment

Environment	Success rate	Extracted equation	Average reward	MSE
Lever-v1	33%	$tau_1 = (0.012 -tau_2 0)\,\frac{d_2}{d_1}$	12.19/20	–
InclinedPlane-v1	21.62%	$F_{\parallel} = mg\sin(\theta)$	- 0.03/0	0
Basketball-v1	3.65%	$y(t) = v_0\sin(\theta)t -4.9t^2 - 0.082t -0.016v_0\sin(\theta) +0.0007$	42.0/100	0.12
Lagrange-v1	6.72%	$L_x^4 = -0.2\cdot (d - 0.48R - 1.21)$	0.93/1	0.056
Lagrange-v1	6.72%	$L_y^4 = 1.01 \cdot\frac{\sqrt{3}}{2}\cdot R$	0.93/1	$3\cdot10^{-10}$
SIRV-v1	48.45%	$V = 1 - \frac{recoveryrate}{transmissionrate}$	- 0.23/0	30.2
Dropfriction-v1	76.38%	$F_f = 4.25\cdot d^2$	4.2 / 10	$2.75\cdot10^{-10}$

The success rate is the percentage of times the threshold R^* was reached

Table 4 Results for Threshold-and-Save on different environments, employing `gplearn` for the extraction and `SAC` for the exploration of the environment

Environment	Extracted equation	MSE
InclinedPlane-v1	$F_{\parallel} = mg\sin(\theta)$	0
Basketball-v1	$y(t) = 0.087\cdot v_0\sin(\theta)\cdot(g-t)\cdot sin(t)$	0.09
Lagrange-v1	$L_x^4 = 0.184R - 0.184d^2 + 0.14$	0.054
Lagrange-v1	$L_y^4 = 0.993\cdot\frac{\sqrt{3}}{2}\cdot R$	$1.5\cdot10^{-5}$
SIRV-v1	$V = \frac{transmissionrate - recoveryrate}{transmissionrate}$	30.2
Dropfriction-v1	$F_f = w$	$1.53\cdot10^{-5}$

We refer to Table 3 for the reward values obtained by SAC

Optionally, the data table resulting from the thresholding function may be preprocessed further, by e.g. only keeping episodes that resulted in the 99% percentile of reward. Clearly, Threshold-and-Save relies on the fact that i) in Context 1, rewards are available and direct the agents towards interesting states; ii) in Context 1, the state space is tabular and contains the variables relevant in describing the phenomenon. When one or both of these conditions are not fulfilled, Threshold-and-Save is not applicable. We report experimental results for several environments in Table 3. We employed the Soft-Actor Critic (Haarnoja et al., 2018) implementation available in `Stable-Baselines3` (Raffin et al., 2021) to optimize the cumulative reward, and `PySR` (Cranmer, 2023) to perform the final equation discovery task. Table 3 shows that with Threshold-and-Save it is possible to gather high reward episodes

Table 5 Results for Threshold-and-Save on different environments employing A2C for the exploration and `PySR` for the extraction of the equation

Environment	Success rate	Extracted equation	Average reward	MSE
InclinedPlane-v1	0.42%	$F_{\parallel} = \sin(\sin(\sin(\sin(\sin(m * 0.013) * 2.3) * 1.94)))) * 0.166$	- 0.127/0	4951
Basketball-v1	0%	-	0/100	–
Lagrange-v1	33%	$L^4_x = 0.01$	0.34/1	0.0007
Lagrange-v1	33%	$L_y^4 = 0.01$	0.34/1	0.41
SIRV-v1	6.67%	$V = 1.03 - \frac{recoveryrate}{0.87\,(transmissionrate - 0.036)} + - \frac{4.21 \cdot 10^{-5}}{recoveryrate}$	- 6.74 /0	6515
Dropfriction-v1	70.62%	$F_f = 4.42 \cdot d^2$	2.93/10	$2.06 \cdot 10^{-10}$

The success rate is the percentage of times the threshold R^* was reached

Table 6 Results for Threshold-and-Save on different environments employing A2C for the exploration and `gplearn` for the extraction of the equation

Environment	Extracted equation	MSE
InclinedPlane-v1	$F_{\parallel} = 0.109$	4924
Lagrange-v1	$L_x^4 = \frac{0.013d}{R^2}$	0.0015
Lagrange-v1	$L^4_y = 0.0097$	0.44
SIRV-v1	V = 0.977 - 2.52 recoveryrate	6390
Dropfriction-v1	F_f = w	$1.52 \cdot 10^{-5}$

We refer to Table 5 for the reward values obtained by SAC

and extract correct equations or equations resembling the correct form. To facilitate this, the `PySR` model was guided by restricting its allowed operators and variables to those present in the correct equation. For the `Lagrange-v1` environment, the variable d was computed during a preprocessing step, as described in section 5. For `Basketball-v1` it was necessary to further assist the model, by computing $v_0 \sin(\theta)$ in advance. Table 4 shows the extracted equations using `gplearn` (XXXX) to perform symbolic regression, following the same guidance strategy outlined above for `PySR`. We additionally employed the A2C algorithm (Mnih et al., 2016) to explore the same environments, whose results we report in Table 5 (`PySR` for extraction) and Table 6 (`gplearn`).

6.3.1 Baseline results and discussion

We observe that when employing Soft-Agent Critic, Threshold-and-Save is able to find equations which obtain a low error, even if the equation's textual representation is not exactly equivalent to the textbook one (Table 1). In particular, we observe that `PySR` is able to recover equations which are lexicographically similar to Table 1 for `Lever-v1`, `InclinedPlane-v1`, `Basketball-v1`, `Lagrange-v1`, and `SIRV-1` (for the

dimension y, second-to-last row in Table 3). The equations are almost identical, up to small additive (0.012 or 1.21) or multiplicative (1.01) constants that are unnecessary. For `Basketball-v1`, the method is able to recover the equation for projectile motion in its main component $y(t) = v_0 sin(\theta)t$, but is not able to generalize to the negative gravity component $-gt$. Similar results, even lexicographically, hold for `gplearn` (Table 4). In `Dropfriction-v1`, both equation finders discover a short equation, while the one from *PySR* has a lower error. More complex equations, as described in section 5, are only needed when multiple material-fluid combinations are modeled.

The A2C algorithm, on the other hand, struggles with obtaining high-quality experimental data which can then be regressed (Tables 5 and 6). We conjecture that the stochasticity of Soft-Agent Critic helps it in exploring the experimental space and thus gathering data, something that is not immediately available for A2C. Using `Science-Gym`, it is possible to explore the capabilities of various combinations of reinforcement learning and symbolic regression/equation discovery algorithms, in such a way and beyond (see Sect. 6.2).

To summarize our results, we observe the following:

- In the full supervision setting, RL exploration is able to recover relevant experimental data, with 6 experiments recovering equations that are lexicographically similar to the ones available in common textbooks, up to small constant variations.
- Exploration and stochasticity in the Soft-Agent Critic algorithm are critical to the success of the full supervision baseline.

Thus, we conclude that a strong baseline is available for the tasks of scientific discovery implemented in `Science-Gym`, which future developments may be compared to.

6.4 Sparse and noisy rewards: an open challenge

We include here an exploration of the feasibility of obtaining explanations for the systems included in `Science-Gym` in more challenging contexts compared to the ones presented previously in this section. In particular, we investigate the scope of obtained explanations when employing *noisy* or *sparse* rewards in conjunction with the straightforward Threshold-And-Save strategy presented in Sect. 6.3. The rationale here is to establish whether it is pos-

Table 7 Obtained equations with `SAC` and `PySR` on the `noisy` reward setting

Environment	Equation	MSE (noisy)	MSE (classic)
InclinedPlane-v1	$m * sin(\theta)$	226637.658	0
Basketball-v1	$t * (-t * (g$ $+v \cdot sin(\theta))$ $+v \cdot sin(\theta))$	8.441	0.12
Lagrange-v1 (L^4_x position)	$0.495 - 0.0287 \cdot d$	0.205	0.056
Lagrange-v1 (L^4_y position)	$0.588 \cdot \sqrt{R}$	0.217	$3 \cdot 10^{-5}$
SIRV-v1	$-\frac{recovery_rate}{transmission_rate}$ $+1.001$	< 0.0001	30.2
Dropfriction-v1	$F_f = 4.16 \cdot d^2$	$2.84 \cdot 10^{-10}$	$2.75 \cdot 10^{-10}$

Table 8 Obtained equations with SAC and PySR on the sparse reward setting

Environment	Equation	MSE (sparse)	MSE (classic)
InclinedPlane-v1	$m * sin(\theta)$	241362.172	0
Basketball-v1	-	–	0.12
Lagrange-v1 (L^4_x position)	0.008·R + 0.473	0.335	0.056
Lagrange-v1 (L^4_y position)	0.1214·R + 0.416	0.984	$3 \cdot 10^{-5}$
SIRV-v1	recovery_rate/transmission_rate	0.351	30.2
Dropfriction-v1	F_f = vel·w·(cos vel − cos d)	$2.15 \cdot 10^{-10}$	$2.75 \cdot 10^{-10}$

sible to still recover scientific information when unreliable or very little supervision is given to a scientific agent. In particular, we set up the following two experiments:

- *Noisy Rewards*: We modify the reward function R into $R' = R + \mathcal{N}(0, \sigma)$ where $\mathcal{N}$ is a Gaussian distribution. We set σ to be between 1 to 10% of the original reward range $|max(R) - min(R)|$,[1].
- *Sparse Rewards*: We binarize the reward function by computing $R'' = \mathbb{I}[R \geq 0.95 \cdot max(R)]$ where $\mathbb{I}$ is the indicator function. That is, only near-to-perfect experiments will receive rewards.

Our expectation for the noisy reward experiments is that the quality of the recovered equation will go down significantly, as more and more uninteresting experiments will seem relevant data, challenging the equation discovery stage of our baseline presented in Sect. 6.3. In terms of sparse rewards, we expect the RL exploration to become more challenging and less useful data to be recovered.

We employ SAC for the exploration/exploitation of the environments and PySR to recover equations from the data, as this combination proved most successful in the full supervision scenario. We present results for these experiments in Tables 7 and 8.

We contextualize the discussion of the results by prefacing that autonomous scientific discovery with no rewards is a completely open task, with no currently known system displaying this capacity without significant human intervention at runtime or bootstrapping of their "knowledge base" (Kramer et al., 2023), something that is not explicitly provided in Science-Gym. Our experiments confirm the state of the field: currently, supervision is critical to the success of AI-driven scientific discovery. In particular, we observe that in the noisy reward settings (Table 7) our baseline has limited success in discovering the equation governing the system: while SIRV-v1 is still a success and Lagrange-v1 is correct up to the difference of a constant and squared-root term when recovering the y position of the satellite, the same cannot be said for the other environments. Here, we observe that the experimental data deemed relevant by simple thresholding markedly goes down in quality, that is, it might represent irrelevant states as well. This impacts the error of the recovered equation significantly.

The sparse reward setting (Table 8) is even more punitive to our simple baseline, as it appears to get lexicographically close to the desired equation only on SIRV-v1. Here, we observe that the reinforcement learning stage lacks exploration: The agent learns to solve only the simplest instances of the problem, and the experimental data lacks variance.

[1] We set the noise level to 10% for Lagrange-v1 SIRV-v1 and DropFriction-v1. We set the noise level to 5% for Basketball-v1 and InclinedPlane-v1.

Table 9 Main characteristics of `BoxingGym` (Gandhi et al., 2025) and `Science-Gym`, compared

Characteristics	`BoxingGym`	`Science-Gym`
Type of scientific problem	Probabilistic equations	Deterministic equations
Input into experiments	Arrays with parameters	Arrays with parameters
Focus of the project	Prompt generation, and LLM answer parsing	Selecting experiments, and equation discovery
Dependencies	Commercial LLMs (OpenAI or Claude)	Open source
Agents	LLMs	RL algorithms
Explanation	Natural language and programs (pymc)	Table of interesting measurements and equations
Metric	New agent prediction after explanation	Error on test data

Table 10 Results for the qualitative comparison of `BoxingGym` and `Science-Gym` on `SIRV-v1`

	`BoxingGym`	`Science-Gym`
Training experiments	15 (39,576 Token used)	20,000
Testing experiments	10 (4801 Token used)	200
Time	44.28 (train) + 46.22 (test)	50.82 (train) + 32.71 (test)
Explanation	'To balance infection and recovery, increase the vaccination rate to reduce susceptible individuals. This decreases new infections, especially when the transmission rate is significantly higher than the recovery rate. Aim for a high vaccination rate to achieve equilibrium between infected and recovered individuals.'	gravity + sin(mass * -0.04016149) * 16.685604 + 493.51007
Mean absolute error	0.406	0.139

Table 11 Results for the qualitative comparison of BoxingGym and `Science-Gym` on `InclinedPlane-v1`

	BoxingGym	`Science-Gym`
Training experiments	15 (33,724 Token used)	5000
Testing experiments	10 (4823 Token used)	200
Time	38.74 (train) + 46.49 (test)	74.02 (train) + 26.78 (test)
Explanation	'To predict the force needed for equilibrium, consider mass, gravity, and tilt angle. Higher mass and gravity require more force. A larger tilt angle may need less force. Start with a moderate force and adjust based on these factors to minimize the distance from the origin.'	gravity + sin(mass * -0.04016149) * 16.685604 + 493.51007
Mean absolute error	408.996	228.797

The equations found may not be wrong per se, and they represent the experimental data with some faithfulness as evidenced by the relatively low error: the final extracted equations, however, completely lack generality and are mostly reduced to a few constant terms (see e.g. `Lagrange-v1`). In another situation, the reinforcement learning stage failed to recover successful episodes (`Basketball-v1`), speaking to the difficulty of obtaining successful experiments when only very sparse rewards are given.

Here, we draw the conclusion that `Science-Gym` provides a significant future challenge for researchers seeking to develop algorithmic advancements in the space of scientific

discovery and experimental design, as our simple baseline does not significantly generalize to very sparse and noisy rewards.

6.5 A qualitative comparison of `Science-Gym` and BoxingGym

One recently released library and testbed to assist in the creation of AI agents for scientific discovery is BoxingGym (Gandhi et al., 2025). BoxingGym relies on large language models' (LLMs) text-generation capabilities to create natural language descriptions of given problem sets and to formalize new experimentation strategies.

The problem sets implemented in BoxingGym and `Science-Gym` are disjoint, and the libraries present significant differences in their assumptions and capabilities. Table 9 provides an overview of both frameworks and the problems implemented in them. In both systems, agents are supposed to interact with a scientific problem and find an explanation. BoxingGym is designed, as mentioned before, for LLM agents, and the framework handles the creation and evaluation of prompts. BoxingGym consists of two phases. In the first phase, a "scientist"-LLM interacts with the problem. For the tilted plate, for example, a prompt could be "Find the force (input) that must be applied to a box so that it does not move on a tilted plane. The state of a system is described by $[mass, gravity, tilt_angle, force_previous_applied]$. The output of the investigated system should be 0." In the second step, after the "scientist"-LLM has interacted with the environment, it summarizes the knowledge gained, and a new agent is supposed to carry out successful experiments based on this explanation.

To illustrate the differences between ScienceGym and BoxingGym, we implement prompts based on `InclinedPlane-v1` and `SIRV-v1` environments in BoxingGym, and present results in Tables 9, 10,and 11.

The main result between the two systems lies in the number of experiments that can be performed in the same amount of time. BoxingGym is based on proprietary LLM models and generates approximately 2500 tokens per experiment in the learning phase. As such, each experiment costs 1.5ct (assuming a price of 5$ per 10^6 tokens). Since ScienceGym does not call an external API and is not based on resource-intensive LLMs, it can perform ~ 300 times more experiments within the same time. The smaller number of experiments also means that the explanations in BoxingGym often have a qualitative nature and can be derived from the background knowledge of the LLM, e.g., "If the box is heavy, more force must be applied,". Sciencegym, on the other hand, provides quantitative explanations through equations.

From this qualitative comparison, we conclude that `Science-Gym` has specific characteristics that make it unique in the space of the currently available discovery testbeds. For one, it requires agents to perform exploration of scientific environments from scratch and without relying on pre-trained, multi-task models such as LLMs, which is also useful in terms of runtime. Moreover, it enforces and evaluates solutions in terms of equations rather than natural language explanations, which appears to us to be more formal than some of the explanations/solutions recovered by BoxingGym. Nonetheless, natural language has of course a high explanatory power, and potential synergies with LLMs in general and BoxingGym are an intriguing point of consideration for future research.

7 Conclusion

Science-Gym presents a novel framework for evaluating AI-driven scientific discovery by providing environments that simulate four classical physics problems and one problem from epidemiology. The library's compatibility with the Gym API makes it accessible and versatile, facilitating integration into existing reinforcement learning pipelines. By offering various contexts of increasing difficulty, Science-Gym allows researchers to systematically test and improve the robustness and adaptability of AI agents.

The potential applications of Science-Gym span multiple fields, including educational technology, automated experiment systems, robotics, autonomous systems, economics, computer vision, environmental science, and advanced AI. The diverse range of environments and the flexibility in observational data and reward structures can make it a valuable tool for developing and benchmarking AI models in different scientific domains. However, Science-Gym has several limitations. Currently, it does not provide a domain-specific language (DSL) for experiment description and analysis. This limitation restricts the agent's ability to generate and interpret complex experimental protocols autonomously. Additionally, Science-Gym imposes a specific action space on the agent, which may not be suitable for all types of scientific discovery tasks. The current framework does not support the discovery of new actions or the integration of embodied agents that need to control actuators, which are essential for tasks requiring physical interactions with the environment. Finally, the current iteration of `Science-Gym` only expects and evaluates equations as the output of the discovery process. However, several complex systems do not allow for summarization via a neat system of equations (Krishnan et al., 2007).

Future work will focus on addressing these limitations by developing or incorporating an DSL, enabling more flexible and sophisticated experimental designs. Furthermore, expanding the action space and supporting the discovery of new actions will enhance the agent's capability to perform more complex and realistic scientific discovery tasks.

In summary, `Science-Gym` offers a novel platform for exploring AI-driven scientific discovery, despite its current limitations. By providing a structured and flexible environment, it opens up new opportunities for research and development in various scientific fields, paving the way for more intelligent and autonomous, and ultimately also, hopefully, more useful AI systems.

Author contributions M.C., N.S., L.B. and S.K. conceptualized the work. M.C., J.B., L.B. and N.S. ran the experiments presented in the paper. E.F, S.J., L.M., F.P.P., P.P., B.R., J.S., J.B. and P.W. contributed simulations (i.e. code) for the software library. M.C. contributed most of the writing, with contributions from L.B., S.K. and J.B.. Additional expertise and data on the drop friction task was provided by S.S..

Funding Open Access funding enabled and organized by Projekt DEAL.

Data availability Data and codebase will be published subject to acceptance of the present manuscript. The experimental suite and library is currently available at github.com/pibborn/science-gym.

Declarations

Conflict of interest The authors declare no Conflict of interest.

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as

you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit http://creativecommons.org/licenses/by/4.0/.

References

Biewald, L. (2020). Experiment Tracking with Weights and Biases. Software available from wandb.com . https://www.wandb.com/

Catto, E.: Box2D. GitHub (2021).

Cohn, D. A., Ghahramani, Z., & Jordan, M. I. (1996). Active learning with statistical models. *Journal of Artificial Intelligence Research, 4*, 129–145.

Cranmer, M. (2023). Interpretable Machine Learning for Science with PySR and SymbolicRegression.jl https://doi.org/10.48550/ARXIV.2305.01582 . Publisher: arXiv Version Number: 2. Accessed 2023-05-05

Furmidge, C. (1962). Studies at phase interfaces i. the sliding of liquid drops on solid surfaces and a theory for spray retention. *Journal of Colloid Science, 17*(4), 309–324.

Gandhi, K., Li, M.Y., Goodyear, L., Li, L., Bhaskar, A., Zaman, M., & Goodman, N.D. (2025). BoxingGym: Benchmarking Progress in Automated Experimental Design and Model Discovery. https://github.com/kanishkg/boxing-gym

gplearn. https://github.com/trevorstephens/gplearn. Accessed: 2025-04-22

Haarnoja, T., *et al.* (2018). Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. In: International Conference on Machine Learning, pp. 1861–1870 . PMLR

Jansen, P.e.a. (2024). DiscoveryWorld: A Virtual Environment for Developing and Evaluating Automated Scientific Discovery Agents.

Johnson, J. & *et al.* (2017). Clevr: A diagnostic dataset for compositional language and elementary visual reasoning. In: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), pp. 2901–2910.

Jumper, J., Evans, R., Pritzel, A., Green, T., Figurnov, M., Ronneberger, O., Tunyasuvunakool, K., Bates, R., Žídek, A., Potapenko, A., Bridgland, A., Meyer, C., Kohl, S. A. A., Ballard, A. J., Cowie, A., Romera-Paredes, B., Nikolov, S., Jain, R., Adler, J., … Hassabis, D. (2021). Highly accurate protein structure prediction with alphafold. *Nature, 596*(7873), 583–589. https://doi.org/10.1038/s41586-021-03819-2

Kitano, H. (2021). Nobel turing challenge: creating the engine for scientific discovery. NPJ Systems Biology and Applications **7**(29).

Kawasaki, K. (1960). Study of wettability of polymers by sliding of water drop. *Journal of Colloid Science, 15*(5), 402–407.

Koza, J. R. (1994). Genetic programming as a means for programming computers by natural selection. *Statistics and Computing, 4*(2), 87–112.

Kramer, S., Cerrato, M., Džeroski, S., & King, R. (2023). Automated scientific discovery: From equation discovery to autonomous discovery systems.

Krishnan, A., Giuliani, A., & Tomita, M. (2007). Indeterminacy of reverse engineering of gene regulatory networks: the curse of gene elasticity. *PLoS One, 2*(6), 562.

La Cava, W., Spector, L., & Danai, K. (2021). Srbench: A living benchmark for symbolic regression. arXiv preprint arXiv:2102.13031

Langley, P. (1977). Bacon: A production system that discovers empirical laws. In: Proceedings of the 5th International Joint Conference on Artificial Intelligence (IJCAI), pp. 344–350.

Langley, P.W., Simon, H.A., Bradshaw, G.L., & Zytkow, J.M. (1987). Scientific Discovery: Computational Explorations of the Creative Processes. MIT Press, ???

Li, X., Bodziony, F., Yin, M., Marschall, H., Berger, R., & Butt, H.-J. (2023). Kinetic drop friction. *Nature Communications,14*(1), 4571. https://doi.org/10.1038/s41467-023-40289-8. Number: 1 Publisher: Nature Publishing Group. Accessed 2023-12-12

Luz, P., Struchiner, C., & Galvani, A. (2010). Modeling transmission dynamics and control of vector-borne neglected tropical diseases. *PLoS Neglected Tropical Diseases*. https://doi.org/10.1371/journal.pntd.0000761

Makke, N., S., C. (2024). Interpretable scientific discovery with symbolic regression: A review. Artificial Intelligence Review **57**(2).

Microsoft Research AI4Science, M.A.Q. (2023). The impact of large language models on scientific discovery: a preliminary study using gpt-4. arXiv preprint arXiv:2311.07361

Mnih, V., Badia, A.P., Mirza, M., Graves, A., Lillicrap, T., Harley, T., Silver, D., & Kavukcuoglu, K. (2016). Asynchronous methods for deep reinforcement learning. In: Balcan, M.F., Weinberger, K.Q. (eds.) Proceedings of The 33rd International Conference on Machine Learning. Proceedings of Machine Learning Research, vol. 48, pp. 1928–1937. PMLR, New York, New York, USA . https://proceedings.mlr.press/v48/mniha16.html

Olson, R.S., Bartley, N., Urbanowicz, R.J., & Moore, J.H. (2017). Pmlb: A large benchmark suite for machine learning evaluation and comparison. In: Gecco, pp. 503–510 . ACM

Raffin, A., Hill, A., Gleave, A., Kanervisto, A., Ernestus, M., & Dormann, N. (2021). Stable-baselines3: Reliable reinforcement learning implementations. *Journal of Machine Learning Research, 22*(268), 1–8.

Ross, K., et al. (2009). The automation of science. *Science, 324*(5923), 85–89.

Shuaiwen, L.S., et al. (2023). Deepspeed4science initiative: Enabling large-scale scientific discovery through sophisticated AI system technologies.

Strogatz, S. H. (1994). *Nonlinear dynamics and chaos: With applications to physics, biology, chemistry, and engineering*. Westview Press.

Sutton, R.S., & Barto, A.G. (2018). Reinforcement learning: An introduction. MIT Press.

Udrescu, S.-M., & Tegmark, M. (2020). AI Feynman: A physics-inspired method for symbolic regression. Science Advances **6**(16).

Wang, R., Jansen, P., Côté, M.-A., & Ammanabrolu, P. (2022). ScienceWorld: Is your Agent Smarter than a 5th Grader? . https://arxiv.org/abs/2203.07540

Yi, K., *et al.* (2020). Clevrer: Collision events for video representation and reasoning. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pp. 7661–7670.

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Authors and Affiliations

Mattia Cerrato[1] · Lennart Baur[1] · Jannis Brugger[2] · Sajjad Shumaly[3] · Nicholas Schmitt[4] · Edward Finkelstein[5] · Selina Jukic[1] · Lars Münzel[1] · Felix Peter Paul[1] · Pascal Pfannes[1] · Benedikt Rohr[1] · Julius Schellenberg[1] · Philipp Wolf[1] · Stefan Kramer[1]

✉ Mattia Cerrato
mcerrato@uni-mainz.de

✉ Stefan Kramer
kramerst@uni-mainz.de

1 Johannes Gutenberg University Mainz, Mainz, Germany

2 TU Darmstadt and hessian.AI, Darmstadt, Germany

3 MPI for Polymer Research, Mainz, Germany

4 University of Tübingen, Tübingen, Germany

5 UC Irvine, Irvine, CA, USA